

WindowBending

Nonlinear Deformation Analysis of Pressurized Circular Windows

User Manual

July 7, 2026

Contents

1	Introduction	2
2	Theory	2
2.1	Geometry and assumptions	2
2.2	Linear (small-deflection) plate theory	2
2.3	Large-deflection Föppl–von Kármán equations	3
2.4	Range of validity	3
2.5	Mindlin–Reissner (shear-deformable) theory	4
2.6	Stress recovery	4
2.7	Peak stress and safety factor	5
3	Numerical Implementation	5
3.1	Reformulation for numerical conditioning	5
3.2	First-order state-space system	6
3.3	Solver settings	6
3.4	Thin-plate convergence and graceful fallback	6
4	Software Architecture	7
4.1	Data classes	7
4.2	Key functions	8
5	Installation	8
6	Command-Line Usage	8
6.1	Options	8
6.2	Material presets	8
7	Examples	12
7.1	Default single case	12
7.2	Simply supported edge	13
7.3	Shear-deformable (Mindlin) thick plate	13
7.4	Selecting a material preset	13
7.5	Thickness design sweep	13
7.6	Diameter design sweep	14

7.7	Pressure design sweep	14
7.8	Combined thickness and diameter sweep	14
7.9	Programmatic use	15
7.10	Custom material	15
8	Output Quantities	15
9	Validation and Testing	16
10	Modelling Assumptions and Limitations	16
11	References	16

1 Introduction

WindowBending is a Python tool that computes the structural response of a circular window (for example, an optical fused-silica window) subjected to a uniform pressure load on one face. It solves the axisymmetric, large-deflection Föppl–von Kármán plate equations and reports:

- the transverse deflection profile $w(r)$ and the centre deflection;
- the radial and circumferential (hoop) stress distributions on both faces;
- the peak tensile stress; and
- the safety factor against a user-supplied allowable stress.

The tool supports two edge support models (*clamped* and *simply supported*) and an optional *design sweep* that varies one parameter (thickness or pressure) over a range and tabulates the resulting deflection, stress, and safety factor.

Motivating problem. The default configuration models a 450 mm diameter, 25 mm thick fused-silica window, fixed around its outer edge, with a uniform pressure of 101,000 Pa (1 atm) applied to the top face.

2 Theory

2.1 Geometry and assumptions

The window is modelled as a thin, flat, circular plate of radius a and uniform thickness t , made of a linear-elastic, isotropic material with Young’s modulus E and Poisson’s ratio ν . Because both the geometry and the loading are axisymmetric, all field quantities depend only on the radial coordinate $r \in [0, a]$. The transverse deflection is denoted $w(r)$ (positive in the direction of the applied pressure) and the in-plane radial displacement is $u(r)$.

2.2 Linear (small-deflection) plate theory

For small deflections ($w \ll t$), Kirchhoff–Love plate theory gives a closed-form centre deflection for a uniformly loaded circular plate. The flexural rigidity is

$$D = \frac{E t^3}{12(1 - \nu^2)}. \quad (1)$$

For a **clamped** edge,

$$w_0^{\text{lin}} = \frac{p a^4}{64 D}, \quad (2)$$

and for a **simply supported** edge,

$$w_0^{\text{lin}} = \frac{p a^4 (5 + \nu)}{64 D (1 + \nu)}. \quad (3)$$

These expressions are used both as a sanity baseline and to seed the nonlinear solver. They are exposed by `PlateConfig.linear_center_deflection()`.

2.3 Large-deflection Föppl–von Kármán equations

When the deflection becomes a non-negligible fraction of the thickness, the plate develops in-plane (membrane) stretching that stiffens the response. This coupling is captured by the axisymmetric Föppl–von Kármán equations. The membrane stiffness is

$$C = \frac{E t}{1 - \nu^2}, \quad (4)$$

and the radial and circumferential membrane stress resultants are

$$N_r = C \left(u' + \nu \frac{u}{r} + \frac{1}{2} (w')^2 \right), \quad (5)$$

$$N_\theta = C \left(\frac{u}{r} + \nu u' + \frac{1}{2} \nu (w')^2 \right), \quad (6)$$

where a prime denotes d/dr . The coupled governing equations are the transverse equilibrium equation,

$$D \nabla^4 w = p + N_r w'' + N_\theta \frac{w'}{r}, \quad (7)$$

and the in-plane equilibrium equation,

$$\frac{d}{dr}(N_r) + \frac{N_r - N_\theta}{r} = 0, \quad (8)$$

where the axisymmetric biharmonic operator acting on w is

$$\nabla^4 w = w'''' + \frac{2}{r} w''' - \frac{1}{r^2} w'' + \frac{1}{r^3} w'. \quad (9)$$

2.4 Range of validity

The Kirchhoff and Föppl–von Kármán models rest on two independent assumptions, each with its own dimensionless ratio:

- **Thin-plate (shear-free) assumption:** the thickness must be small relative to the in-plane span, quantified by the thickness-to-diameter ratio t/D (equivalently the radius-to-thickness ratio $R/h = 1/(2t/D)$, with $R = a$ the radius and $h = t$ the thickness). Kirchhoff–Love kinematics neglect transverse shear and are accurate for genuinely thin plates, $t/D < 0.05$ ($R/h > 10$). Beyond this the plate carries an appreciable shear deflection that Kirchhoff theory omits: a *transition zone* $0.05 \leq t/D < 0.1$ ($5 < R/h \leq 10$) where the Mindlin–Reissner model is recommended for accuracy, and a definite thick-plate region $t/D \geq 0.1$ ($R/h \leq 5$) where it is required.
- **Moderate-rotation assumption:** the Föppl–von Kármán strain measure retains only the leading nonlinear term $\frac{1}{2}(w')^2$, which is accurate while the central deflection is at most a few thicknesses ($w_0/t \lesssim 1$ – 5) and edge rotations stay below about 10° to 15° .

For the windows of interest the deflection stays far below the thickness ($w_0/t \leq 0.06$), so the moderate-rotation limit is comfortably satisfied and the response is effectively quasi-linear. The binding limitation is instead the thin-plate assumption: much of the design space—including the default 450 mm diameter, 25 mm thick window ($t/D = 0.056$, $R/h = 9$)—falls in the transition zone or beyond (t/D reaches 0.14 at the thick, small-diameter corners). There the shear-free Kirchhoff model under-predicts the deflection, motivating the shear-deformable model of the next subsection.

2.5 Mindlin–Reissner (shear-deformable) theory

When t/D exceeds the thin-plate limit the tool can solve the axisymmetric *Mindlin–Reissner* plate equations, which relax the Kirchhoff constraint by letting the cross-section rotation $\beta(r)$ be independent of the slope w' . The difference between them is the transverse-shear strain, resisted by the shear stiffness

$$k_s = \kappa G t, \quad G = \frac{E}{2(1+\nu)}, \quad \kappa = \frac{5}{6}, \quad (10)$$

with κ the shear-correction factor for a homogeneous rectangular section. The transverse shear resultant is $Q = k_s (w' + \beta)$, and the bending moments are expressed through β rather than w'' :

$$M_r = D \left(\beta' + \nu \frac{\beta}{r} \right), \quad M_\theta = D \left(\nu \beta' + \frac{\beta}{r} \right). \quad (11)$$

Combined with the same membrane (von Kármán) coupling used above, this yields a shear-deformable BVP that reduces to the Kirchhoff solution as $t/D \rightarrow 0$ (where $k_s \rightarrow \infty$ forces $\beta \rightarrow -w'$). For a clamped plate the linear limit gives the closed-form central deflection

$$w_{\text{Mindlin}}(0) = \underbrace{\frac{p a^4}{64 D}}_{\text{bending}} + \underbrace{\frac{p a^2}{4 \kappa G t}}_{\text{shear}}, \quad (12)$$

i.e. the Kirchhoff deflection plus an additive shear term. The peak bending stress is essentially unchanged (transverse equilibrium fixes $Q = -pr/2$ in both theories); only the deflection grows, by the shear compliance in (12). The theory is selected with the `--plate-theory` flag (Section 6); `auto` switches to Mindlin whenever $R/h \leq 10$ (equivalently $t/D \geq 0.05$), so the transition zone and thicker plates use the shear-deformable model, and Kirchhoff is retained only for genuinely thin plates ($R/h > 10$).

Three conditions follow from axisymmetry at the centre $r = 0$:

$$w'(0) = 0, \quad w'''(0) = 0, \quad u(0) = 0. \quad (13)$$

At the supported edge $r = a$, the deflection vanishes and the edge is radially constrained:

$$w(a) = 0, \quad u(a) = 0. \quad (14)$$

The remaining condition distinguishes the two support types:

- **Clamped:** zero edge rotation,

$$w'(a) = 0. \quad (15)$$

- **Simply supported:** zero radial bending moment,

$$M_r(a) = -D \left(w''(a) + \nu \frac{w'(a)}{a} \right) = 0. \quad (16)$$

2.6 Stress recovery

Once $w(r)$ and $u(r)$ are known, the bending moments per unit length are

$$M_r = -D \left(w'' + \nu \frac{w'}{r} \right), \quad M_\theta = -D \left(\frac{w'}{r} + \nu w'' \right). \quad (17)$$

The total surface stress superposes a membrane part (uniform through thickness) and a bending part (linear through thickness, extreme at the faces):

$$\sigma_r^{\text{membrane}} = \frac{N_r}{t}, \quad \sigma_\theta^{\text{membrane}} = \frac{N_\theta}{t}, \quad (18)$$

$$\sigma_r^{\text{bending}} = \frac{6M_r}{t^2}, \quad \sigma_\theta^{\text{bending}} = \frac{6M_\theta}{t^2}. \quad (19)$$

The stresses on the two faces are therefore

$$\sigma^{\text{top}} = -\sigma^{\text{bending}} + \sigma^{\text{membrane}}, \quad (20)$$

$$\sigma^{\text{bottom}} = +\sigma^{\text{bending}} + \sigma^{\text{membrane}}. \quad (21)$$

2.7 Peak stress and safety factor

The peak tensile stress is the maximum over the radial coordinate of all four stress components (radial/hoop on top/bottom faces):

$$\sigma_{\text{max}} = \max_r \{ \sigma_r^{\text{top}}, \sigma_\theta^{\text{top}}, \sigma_r^{\text{bottom}}, \sigma_\theta^{\text{bottom}} \}. \quad (22)$$

The safety factor compares this against the user-supplied allowable stress σ_{allow} :

$$\text{SF} = \frac{\sigma_{\text{allow}}}{\sigma_{\text{max}}}. \quad (23)$$

Because fused silica is brittle, the relevant allowable stress is its “allowable design stress” as listed in the reference glass-property table. The default allowable stress is 4.69 MPa (680 psi).

3 Numerical Implementation

3.1 Reformulation for numerical conditioning

A direct first-order reduction of the fourth-order biharmonic operator carries $1/r^2$ and $1/r^3$ singular terms near the centre. In practice these make the collocation solver `scipy.integrate.solve_bvp` ill-conditioned: it fails to converge and returns a deflection biased low by roughly 7%, even in the linear limit where the answer must match Eqs. (2) and (3).

To avoid this, the transverse equation is rewritten using the Laplacian intermediate variable

$$M \equiv \nabla^2 w = w'' + \frac{w'}{r}. \quad (24)$$

This leaves only mild $1/r$ singularities. The simply-supported moment condition becomes, using $w'' = M - w'/r$,

$$M(a) = (1 - \nu) \frac{w'(a)}{a}. \quad (25)$$

With this reformulation the solver reproduces both closed-form linear solutions to ten significant digits.

3.2 First-order state-space system

The problem is cast as a first-order boundary-value problem in the state vector

$$\mathbf{y} = [w, w', M, M', u, u']^T, \quad (26)$$

integrated over $r \in [r_0, a]$, where $r_0 = a \times 10^{-6}$ is a small offset that regularizes the residual $1/r$ terms at the centre (constant `CENTER_REGULARIZATION`). The six boundary residuals encode the three symmetry conditions at r_0 and the three edge conditions at a .

3.3 Solver settings

The nonlinear BVP is solved with `solve_bvp`, seeded with the clamped small-deflection shape from Eq. (2). Key defaults are listed in Table 1.

Table 1: Default solver configuration constants.

Constant	Value	Meaning
<code>DEFAULT_MESH_NODES</code>	250	Initial mesh nodes
<code>DEFAULT_OUTPUT_POINTS</code>	400	Output sample points
<code>SOLVER_TOLERANCE</code>	10^{-7}	Collocation residual tolerance
<code>SOLVER_MAX_NODES</code>	50,000	Maximum adaptive mesh nodes
<code>CENTER_REGULARIZATION</code>	10^{-6}	Centre offset r_0/a
<code>KIRCHHOFF_CONTINUATION_STEPS</code>	10	Load-continuation steps on failure
<code>LINEAR_FALLBACK_WT_RATIO</code>	0.2	w_0/t below which linear fallback is used
<code>VON_KARMAN_WT_WARN_RATIO</code>	5	w_0/t above which validity is warned

3.4 Thin-plate convergence and graceful fallback

As the plate is made thinner, the von Kármán central deflection grows toward the order of the thickness ($w_0/t \gtrsim 1$) and the response becomes membrane-dominated. A sharp bending *boundary layer* then forms at the clamped edge, whose width shrinks with load. A uniform-mesh collocation solver refines that layer without bound and exhausts its node budget; empirically the single-shot solve succeeds for the fused-silica window down to about $t = 6$ mm ($w_0/t \approx 0.5$) and fails below that. This is a strong-nonlinearity limit, not a small-deflection conditioning problem.

The solver therefore uses a three-tier strategy:

1. **Single-shot solve.** The nonlinear BVP is attempted once from the linear initial guess (the common, fast path).
2. **Load continuation.** On failure the pressure is ramped from a small fraction to its full value in `KIRCHHOFF_CONTINUATION_STEPS` steps, each converged solution seeding the next. This homotopy reaches deeper into the nonlinear regime.
3. **Graceful fallback.** If continuation also fails, the response is inspected: when $w_0/t \leq \text{LINEAR_FALLBACK_WT_RATIO}$ (0.2) the membrane coupling is negligible, so the accurate closed-form linear Kirchhoff solution (Section 2.2) is returned with a warning. Otherwise a descriptive `RuntimeError` is raised explaining that the case is in the strongly nonlinear, membrane-dominated regime.

Independently, whenever a converged solution has $w_0/t > \text{VON_KARMAN_WT_WARN_RATIO}$ (5) the tool warns that the Föppl–von Kármán moderate-rotation assumption itself is violated (Section 2.4) and a large-rotation shell model is required. Because such a result is quantitatively unreliable, this breach is surfaced prominently in *every* output channel so it cannot be missed:

- a Python `UserWarning` is emitted;
- the console prints a boxed *MODEL VALIDITY WARNING* banner (before and after a single case; sweeps additionally flag each offending row/cell and list the affected cases);
- the saved figures carry a bold red banner across the bottom (*MODEL INVALID: deflection exceeds Föppl–von Kármán validity*);
- the Excel workbooks gain a red-highlighted validity row, and sweep workbooks additionally shade the offending value cells red.

4 Software Architecture

The implementation lives in a single module, `window_deformation.py`, and is organised around small, immutable data classes and pure functions so that the numerical core is independent of the command-line interface and plotting.

4.1 Data classes

Material

Young’s modulus, Poisson’s ratio, allowable stress, and a name. Validates that the modulus and allowable stress are positive and that $-1 < \nu < 0.5$.

PlateConfig

Geometry (diameter, thickness), pressure, material, and boundary condition. Provides derived properties `radius_m`, `flexural_rigidity`, `membrane_stiffness`, and `linear_center_deflection()`. The convenience constructor `from_engineering_units()` accepts millimetre geometry.

PlateSolution

The computed fields: r , w , the linear and nonlinear centre deflections, the four stress arrays, the peak stress, the safety factor, and the boundary condition. Exposes `center_deflection_mm` and `max_tensile_mpa` convenience properties.

SweepPoint

One evaluated case of a design sweep: the swept value, centre deflection (mm), peak stress (MPa), and safety factor.

GridPoint

One evaluated case of a combined grid sweep: thickness (mm), diameter (mm), thickness/diameter ratio, resolved plate theory, linear and nonlinear centre deflection (mm), peak stress (MPa), and safety factor.

4.2 Key functions

`solve_plate(config, n_mesh, n_points)`

Solves the BVP for a single `PlateConfig` and returns a `PlateSolution`.

`run_design_sweep(base_config, sweep_variable, sweep_values)`

Evaluates the plate response across a range of thickness or pressure and returns a list of `SweepPoint`.

`run_sweep_grid(base_config, thickness_values_mm, diameter_values_mm)`

Evaluates the response over a full thickness $\times$ diameter grid (pressure, material and edge fixed) and returns a list of `GridPoint`.

`build_parser(), config_from_args(args)`

Construct the CLI parser and translate parsed arguments into a `PlateConfig`.

`main(argv)`

Entry point: dispatches to single-case or sweep mode, prints results, and optionally plots.

Plotting helpers import `matplotlib` lazily and force the non-interactive Agg backend, so the numerical core has no hard GUI dependency, remains fast to import for testing, and figures are written to disk rather than displayed. JPEG output additionally uses Pillow, which ships as a `matplotlib` dependency.

5 Installation

The tool requires Python 3.10+ and the packages `numpy`, `scipy`, `matplotlib`, and `openpyxl` (for Excel export).

```
python -m pip install numpy scipy matplotlib openpyxl
```

6 Command-Line Usage

The script is run directly with Python. All parameters have defaults matching the fused-silica window described in the introduction.

```
python window_deformation.py [options]
```

6.1 Options

6.2 Material presets

The `--material` flag selects a named material preset that sets Young's modulus E , Poisson's ratio ν and the allowable stress in one step. The built-in presets are listed in Table 3. Run `--list-materials` to print the same table from the command line and exit.

The individual `--youngs-modulus-pa`, `--poisson-ratio` and `--allowable-stress-mpa` flags override the corresponding preset value, so a preset can be used as a starting point and then tuned. For example, `--material sapphire --allowable-stress-mpa 200` keeps sapphire's stiffness but applies a more conservative allowable stress. The selected material name is echoed in the console header, the figure subtitles and the spreadsheet parameter rows.

Table 2: Command-line options.

Option	Default	Description
<code>--diameter-mm</code>	450	Window diameter (mm)
<code>--thickness-mm</code>	25	Window thickness (mm)
<code>--pressure-pa</code>	101000	Uniform pressure (Pa)
<code>--material</code>	<code>fused_silica</code>	Named material preset (see Table 3)
<code>--list-materials</code>	<code>off</code>	Print the preset table and exit
<code>--youngs-modulus-pa</code>	<code>preset</code>	Override the preset Young's modulus (Pa)
<code>--poisson-ratio</code>	<code>preset</code>	Override the preset Poisson's ratio
<code>--allowable-stress-mpa</code>	<code>preset</code>	Override the preset allowable stress (MPa)
<code>--boundary-condition</code>	<code>clamped</code>	<code>clamped</code> or <code>simply_supported</code>
<code>--plate-theory</code>	<code>auto</code>	<code>kirchhoff</code> , <code>mindlin</code> , or <code>auto</code> (Mindlin when $R/h \leq 10$, i.e. $t/D \geq 0.0$)
<code>--sweep-variable</code>	<code>none</code>	<code>none</code> , <code>thickness_mm</code> , <code>pressure_pa</code> , <code>diameter_mm</code> , or <code>all</code>
<code>--sweep-start</code>	<code>—</code>	Sweep start value
<code>--sweep-stop</code>	<code>—</code>	Sweep stop value
<code>--sweep-count</code>	7	Number of sweep samples
<code>--output-dir</code>	<code>figures</code>	Directory for saved figures and spreadsheets
<code>--figure-name</code>	<code>auto</code>	Base name for figure files (boundary suffix still appended)
<code>--excel-name</code>	<code>auto</code>	Base name for the Excel workbook (boundary suffix still appended)
<code>--figure-format</code>	<code>jpg</code>	One or more of <code>jpg</code> , <code>png</code> , <code>svg</code> , <code>pdf</code> (e.g. <code>jpg svg</code>)
<code>--no-save</code>	<code>off</code>	Skip writing all output files (figures and spreadsheets)
<code>--no-figures</code>	<code>off</code>	Skip only the figures (still write spreadsheets)
<code>--no-excel</code>	<code>off</code>	Skip only the Excel spreadsheets (still write figures)

How to use the preset library. The presets are used in four steps:

1. **Discover.** List the available presets and their properties without running a solve:

```
python window_deformation.py --list-materials
```

2. **Select.** Pass a preset key (the left column of Table 3) to `--material`. All other inputs keep their defaults unless you set them:

```
python window_deformation.py --material beryllium --thickness-mm 15
```

3. **Tune (optional).** Override any single property while keeping the rest of the preset. The override always wins over the preset value:

```
python window_deformation.py --material n_bk7 --allowable-stress-mpa 25
```

4. **Combine.** A preset applies to every case of a design sweep, because the sweep holds the material fixed while varying geometry or load:

```
python window_deformation.py --material sapphire \
    --sweep-variable thickness_mm --sweep-start 10 --sweep-stop 30 --
    sweep-count 5
```

If `--material` is omitted the tool uses `fused_silica`, so existing commands behave exactly as before. Supplying an unknown key produces an `argparse` error that lists the valid choices.

Table 3: Built-in material presets. Values are nominal room-temperature properties. For the glasses the allowable stress is the “allowable design stress” listed directly in the reference glass-property table; for methyl methacrylate (*acrylic*) it follows table Note #8 (design stress limited to the lower of modulus of rupture / 10 or $(9200/10) \times (E/360,000)$), giving 920 psi at 75 °F. Verify against certified data before relying on them for design.

Key	Name	E (GPa)	ν	Allowable (MPa)
<code>fused_silica</code>	Fused Silica	72.0	0.170	4.69
<code>n_bk7</code>	N-BK7	82.0	0.206	3.45
<code>borofloat</code>	Borofloat 33	64.0	0.200	25.00
<code>aluminosilicate</code>	Aluminosilicate (1723)	85.5	0.260	4.55
<code>vykor</code>	96% Silica (Vycor)	68.9	0.190	4.24
<code>pyrex</code>	Borosilicate (Pyrex)	62.7	0.200	4.21
<code>plate_glass</code>	Plate Glass (Herculite)	68.9	0.210	4.48
<code>lead_glass</code>	High-Lead X-Ray Glass	55.2	0.230	3.45
<code>acrylic</code>	Methyl Methacrylate	2.5	0.390	6.34
<code>sapphire</code>	Sapphire	345.0	0.290	350.00
<code>zerodur</code>	Zerodur	90.3	0.243	57.00
<code>caf2</code>	Calcium Fluoride	75.8	0.260	36.00
<code>beryllium</code>	Beryllium	287.0	0.032	240.00

Presets from the Python API. The same presets are available programmatically through the `wd.MATERIAL_PRESETS` dictionary, keyed by the identical names accepted by `--material`:

```

1 import window_deformation as wd
2
3 sapphire = wd.MATERIAL_PRESETS["sapphire"]           # a ready-made Material
4 config = wd.PlateConfig.from_engineering_units(
5     diameter_mm=450.0, thickness_mm=25.0, pressure_pa=101_000.0,
6     material=sapphire,
7 )
8 solution = wd.solve_plate(config)
9 print(solution.center_deflection_mm, solution.max_tensile_mpa)

```

Because `Material` is an immutable (frozen) dataclass, tune a preset by constructing a copy with `dataclasses.replace` rather than mutating it:

```

1 from dataclasses import replace
2
3 conservative = replace(wd.MATERIAL_PRESETS["beryllium"],
4     allowable_stress_pa=200 * wd.PA_PER_MPA)

```

Figure output. The tool never opens an interactive window. Instead it renders results with the non-interactive Agg backend and writes image files into the output directory (`figures/` by default), reporting each saved path on the console. A single case produces `single_case_deflection_clamped.jpg` and `single_case_stress_clamped.jpg`; a single-parameter sweep produces `sweep_variable_clamped.jpg`; the combined sweep produces `sweep_combined_clamped.jpg` (the trailing boundary-condition name is appended automatically—see *Output file naming* below). The figure format defaults to JPEG but is selectable with `--figure-format`, which accepts one or more of `jpg`, `png`, `svg` and `pdf`. Passing

several writes the same figure in each format under a shared name, for example `--figure-format jpg svg` emits both `..._clamped.jpg` and `..._clamped.svg`; `svg` and `pdf` are resolution-independent vector formats suited to publications. Pass `--no-figures` to skip the figures (while still writing spreadsheets), or `--no-save` to skip all output files.

For example, to render each figure as both a JPEG (for quick preview) and an SVG (for inclusion in a report or further editing):

```
python window_deformation.py --figure-format jpg svg
```

which writes, for the default single case, `single_case_deflection_clamped.jpg` and `single_case_deflection_clamped.svg` (plus the matching stress pair). Because the overwrite guard reserves a stem across *all* requested formats at once, re-running the command bumps every format together (`..._clamped_1.jpg` and `..._clamped_1.svg`), so the raster and vector copies of one figure never drift apart.

Spreadsheet output. Alongside every figure the tool writes an Excel workbook (`.xlsx`) containing the same numeric results echoed to the console, into the same output directory. A single case produces `single_case_clamped.xlsx` with two worksheets: *Single Case* (a labelled quantity/value/units summary table) and *Profile Data*, which tabulates the full radial profiles behind the plotted curves—one row per solver sample point giving radius, transverse deflection and the four stress components (radial and hoop stress on the top and bottom faces). A single-parameter sweep produces `sweep_variable_clamped.xlsx` (one row per swept value); and the combined sweep produces `sweep_combined_clamped.xlsx` with four worksheets. The first, *Case Details*, lists every grid case with the same per-case quantities reported for a single case—thickness, diameter, thickness/diameter ratio, resolved plate theory, linear and nonlinear centre deflection, peak tensile stress and safety factor (one row per thickness/diameter combination). The remaining three worksheets—center deflection, peak tensile stress and safety factor—each present that response as a thickness $\times$ diameter matrix. Every combined-sweep worksheet is prefixed with a *parameters held constant* block listing the pressure, boundary condition, plate theory, Young’s modulus, Poisson’s ratio and allowable stress used for the grid. Spreadsheet cells hold full-precision values displayed to five decimals. Every workbook also carries a *Note* row stating the plate-theory selection rule—under `auto`, based on the radius-to-thickness ratio $R/h = 1/(2t/D)$, Kirchhoff thin-plate theory is used for $t/D < 0.05$ ($R/h > 10$) and Mindlin–Reissner thick-plate theory for $t/D \geq 0.05$ ($R/h \leq 10$, spanning the transition zone $0.05 \leq t/D < 0.1$ and the definite thick-plate region $t/D \geq 0.1$), while an explicit `--plate-theory` choice overrides it. Pass `--no-excel` to skip the spreadsheets (while still writing figures), or `--no-save` to skip all output files. Excel export uses `openpyxl`.

Output file naming. Every figure and spreadsheet name always ends with the boundary condition (`name_clamped`, `name_simply_supported`) so results are self-identifying and runs with different edge conditions never collide. The tool also never overwrites an existing output file: if the boundary-condition name is already taken, a numeric suffix is added (`name_clamped_1`, `name_clamped_2`, ...) to guarantee uniqueness. The console always reports the exact path written.

Custom output names. The default base names (`single_case`, `sweep_variable`, `sweep_combined`) can be overridden with `--figure-name` and `--excel-name`. The boundary-condition suffix, uniqueness suffix and file extension are still appended, so `--excel-name results` writes `results_clamped.xlsx`. In single-case mode the two figures take a plot-type suffix, so `--figure-name mywindow` writes `mywindow_deflection_clamped.jpg` and `mywindow_stress_clamped.jpg`. For example,

```
python window_deformation.py --figure-name mywindow --excel-name results
```

names the default single case `mywindow_deflection_clamped.jpg`, `mywindow_stress_clamped.jpg` and `results_clamped.xlsx`.

Diameter sweep defaults. When `--sweep-variable diameter_mm` is selected without explicit `--sweep-start/--sweep-stop`, the sweep defaults to diameters 250 mm to 450 mm in 5 steps (radius 125 mm to 225 mm in 25 mm increments).

Combined sweep. Selecting `--sweep-variable all` evaluates a full thickness $\times$ diameter grid at the fixed base pressure and presents it in a single figure (`sweep_combined_clamped.jpg`) as *families* of curves. The figure is a 2×3 grid of panels: the top row plots each response versus thickness with one curve per diameter, and the bottom row plots each response versus diameter with one curve per thickness. The three columns are center deflection, peak tensile stress and safety factor. The thickness axis defaults to 15 mm to 35 mm in 5 steps and the diameter axis to 250 mm to 450 mm in 5 steps (25 solved cases). The pressure sweep is intentionally excluded so that the whole grid shares one loading condition. Explicit `--sweep-start/--sweep-stop/--sweep-count` values are ignored in this mode. The console prints three matrices (deflection, stress and safety factor) with thickness as rows and diameter as columns.

Held-constant annotations. Every figure states which parameters are held fixed. The single-case deflection and stress figures carry a subtitle listing the full input set (e.g. *Diameter = 450.0 mm, Thickness = 25.0 mm, Pressure = 101,000 Pa, Edge = clamped*), and the console output additionally names the resolved plate theory and the t/D ratio. A single-parameter sweep carries a figure title listing all fixed quantities (e.g. *Held constant: Diameter = 450.0 mm, Pressure = 101,000 Pa, Edge = clamped, Theory = Kirchhoff*). The combined figure places the globally common constants (pressure, edge support and plate theory) in the figure title; each panel's legend identifies the curve family (diameter for the thickness panels, thickness for the diameter panels).

7 Examples

7.1 Default single case

```
python window_deformation.py
```

Produces the console summary below and writes `figures/single_case_deflection_clamped.jpg` and `figures/single_case_stress_clamped.jpg`:

```
=== Fused Silica Window Deformation (von Karman Nonlinear Plate BVP) ===
Diameter: 450.000 mm, Thickness: 25.000 mm, Pressure: 101000.000 Pa
Boundary condition: clamped

Linear center deflection:      0.0419 mm
Nonlinear center deflection:  0.0419 mm
Peak tensile stress:          6.137 MPa
Safety factor (allowable/peak): 0.764
Saved figure: figures\single_case_deflection.jpg
Saved figure: figures\single_case_stress.jpg
```

For this geometry $w/t \approx 0.0017$, so the membrane stiffening is negligible and the nonlinear result coincides with linear theory — as it physically should.

7.2 Simply supported edge

```
python window_deformation.py --boundary-condition simply_supported
```

A simply supported edge permits rotation and therefore deflects substantially more than a clamped edge (≈ 0.187 mm centre deflection for the default geometry).

7.3 Shear-deformable (Mindlin) thick plate

```
python window_deformation.py --diameter-mm 250 --thickness-mm 35 \
    --plate-theory mindlin
```

For a 35 mm plate on a 250 mm aperture the thickness-to-diameter ratio is $t/D = 0.14$, beyond the thin-plate limit. The Mindlin–Reissner solution adds the transverse-shear compliance of (12) and therefore reports a larger centre deflection than the Kirchhoff model, while the peak stress is essentially unchanged. With the default `--plate-theory auto` the same case selects Mindlin automatically because $t/D = 0.14 \geq 0.05$ ($R/h < 10$); passing `kirchhoff` forces the thin-plate model for comparison.

7.4 Selecting a material preset

```
python window_deformation.py --material sapphire
```

Solves the default geometry with the sapphire preset (Table 3); the console header echoes the material name (*Sapphire Window Deformation*). The preset values can be tuned on the same command line—for example, applying a more conservative allowable stress while keeping sapphire’s stiffness:

```
python window_deformation.py --material sapphire --allowable-stress-mpa
    200
```

To see every available preset without running a solve:

```
python window_deformation.py --list-materials
```

7.5 Thickness design sweep

```
python window_deformation.py --sweep-variable thickness_mm \
    --sweep-start 15 --sweep-stop 35 --sweep-count 5
```

Produces an aligned table (and writes `figures/sweep_thickness_mm.clamped.jpg`):

```
=== Design Sweep Summary ===
Sweep variable: Thickness (mm)
```

Value	Deflection(mm)	PeakStress(MPa)	SafetyFactor
15.000	0.19394	17.06445	0.2747
20.000	0.08183	9.59089	0.4888
25.000	0.04389	6.13685	0.7640
30.000	0.02591	4.26132	1.1002
35.000	0.01669	3.13064	1.4976

7.6 Diameter design sweep

With no explicit bounds, the diameter sweep defaults to 250 mm to 450 mm in 5 steps (radius 125 mm to 225 mm):

```
python window_deformation.py --sweep-variable diameter_mm
```

Produces (and writes figures/sweep_diameter_mm.clamped.jpg):

```
=== Design Sweep Summary ===
Sweep variable: Diameter (mm)
```

Value	Deflection(mm)	PeakStress(MPa)	SafetyFactor
250.000	0.00461	1.89379	2.4757
300.000	0.00916	2.72711	1.7192
350.000	0.01654	3.71201	1.2630
400.000	0.02773	4.84855	0.9670
450.000	0.04389	6.13685	0.7640

7.7 Pressure design sweep

```
python window_deformation.py --sweep-variable pressure_pa \
    --sweep-start 50000 --sweep-stop 200000 --sweep-count 6
```

7.8 Combined thickness and diameter sweep

To study both geometry parameters at once under the fixed base pressure, select `all`. Three response matrices are printed (deflection, peak stress and safety factor) and a single combined figure of curve families (figures/sweep_combined.clamped.jpg) is written:

```
python window_deformation.py --sweep-variable all
```

Console output (abbreviated):

```
=== Combined Sweep: Center Deflection (mm) ===
Rows: Thickness (mm) | Columns: Diameter (mm)
Thk\Dia      250.0      300.0      350.0      400.0
            450.0
-----
15.0         0.01950      0.03979      0.07098      0.12108
0.19394
...
35.0         0.00189      0.00365      0.00645      0.01066
0.01669

=== Combined Sweep: Peak Tensile Stress (MPa) ===
... (thickness x diameter matrix) ...

=== Combined Sweep: Safety Factor ===
... (thickness x diameter matrix) ...
```

7.9 Programmatic use

The module can be imported and used directly, which is convenient for scripting or notebooks:

```
1 import window_deformation as wd
2
3 config = wd.PlateConfig.from_engineering_units(
4     diameter_mm=450.0,
5     thickness_mm=25.0,
6     pressure_pa=101_000.0,
7     boundary_condition=wd.CLAMPED,
8 )
9 solution = wd.solve_plate(config, n_points=500)
10
11 print(solution.center_deflection_mm)    # mm
12 print(solution.max_tensile_mpa)        # MPa
13 print(solution.safety_factor)
```

7.10 Custom material

```
1 import window_deformation as wd
2
3 bk7 = wd.Material(
4     youngs_modulus_pa=82e9,
5     poisson_ratio=0.21,
6     allowable_stress_pa=40e6,
7     name="N-BK7",
8 )
9 config = wd.PlateConfig.from_engineering_units(
10     diameter_mm=300.0, thickness_mm=20.0, pressure_pa=101_000.0, material=
11     bk7,
12 )
13 solution = wd.solve_plate(config)
```

The named presets exposed on the command line are also available programmatically through the `wd.MATERIAL_PRESETS` dictionary (keyed by the same names as `--material`), so `wd.MATERIAL_PRESETS["sapphire"]` returns a ready-made `Material`.

8 Output Quantities

Linear center deflection

w_0^{lin} from Eq. (2) or (3); a quick sanity baseline.

Nonlinear center deflection

$w(0)$ from the full von Kármán solve.

Peak tensile stress

σ_{max} , the governing quantity for brittle fracture.

Safety factor

$\sigma_{\text{allow}}/\sigma_{\text{max}}$.

The tool writes its figures as JPEG files by default (no interactive window), and can additionally or alternatively emit PNG, SVG or PDF via `--figure-format`. A single case produces a deflection profile and a stress profile (radial and hoop on both faces); a sweep produces a two-panel figure of deflection versus the swept variable and peak-stress/safety-factor versus the swept variable. Files are named descriptively and written to the output directory (`figures/` by default).

9 Validation and Testing

The package ships with a `pytest` suite (`test_window_deformation.py`) of 37 tests covering:

- input validation and edge cases (non-positive dimensions, out-of-range Poisson ratio, invalid boundary condition, degenerate mesh sizes);
- physical correctness (zero edge deflection, centre is the maximum, monotonic profile, clamped versus simply-supported ordering, thickness and pressure trends);
- agreement of the nonlinear solver with linear theory in the thin-deflection limit and against the closed-form expressions;
- safety-factor scaling and design-sweep behaviour; and
- command-line plumbing.

Run the tests and the linter with:

```
python -m pytest
python -m ruff check .
```

10 Modelling Assumptions and Limitations

- **Plate kinematics.** By default `auto` selects the plate model from the radius-to-thickness ratio: Kirchhoff–Love kinematics (no transverse shear) for genuinely thin plates ($t/D < 0.05$, $R/h > 10$), and the shear-deformable Mindlin–Reissner model (Section 2.5) for the transition zone and thicker ($t/D \geq 0.05$, $R/h \leq 10$). The default window has $t/D = 0.056$ ($R/h = 9$), which lies in the transition zone, so `auto` resolves it to Mindlin; the `--plate-theory` flag can force either model explicitly.
- **Linear-elastic, isotropic material.** No plasticity, viscoelasticity, or anisotropy is modelled.
- **Radially constrained edge.** The edge is fixed against in-plane radial motion ($u(a) = 0$). A free-to-slide edge would reduce membrane stiffening.
- **Uniform static pressure.** The load is a uniform, static pressure on one face. Dynamic, thermal, and self-weight effects are not included.
- **Brittle failure criterion.** The safety factor uses a maximum tensile stress criterion appropriate for brittle materials such as fused silica; it does not account for statistical strength (e.g. Weibull) or surface-flaw distributions.

11 References

1. S. Timoshenko and S. Woinowsky-Krieger, *Theory of Plates and Shells*, 2nd ed., McGraw-Hill, 1959.

2. L. D. Landau and E. M. Lifshitz, *Theory of Elasticity*, 3rd ed., Pergamon Press, 1986 (Föppl–von Kármán equations).
3. P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, 17, 261–272, 2020 (the `solve_bvp` collocation solver).